RELATÓRIO FINAL – API de Oportunidades

Aluno: Leandro Mota Leal

Curso: Análise e Desenvolvimento de Sistemas

Projeto: *API-Oportunidades*

Trello: https://trello.com/b/rGeD3Dyg/projeto-de-extensao-api-de-oportunidades

GitHub: https://github.com/Vermelhow/api-oportunidades.git

VÍDEO DE APRESENTAÇÃO: https://www.youtube.com/watch?v=4GYEsI9nGss

---

1. Introdução

A API de Oportunidades foi desenvolvida como parte do Projeto de Extensão Individual, com o objetivo de criar uma plataforma centralizada para armazenar e disponibilizar oportunidades sociais — como cursos, vagas, eventos, ações voluntárias e iniciativas de apoio à população de baixa renda.
O projeto atende a uma necessidade real de **organização, integração e acessibilidade** das informações, permitindo que outras aplicações (web, mobile ou desktop) consumam esses dados de forma padronizada e segura.

A solução foi construída seguindo boas práticas de desenvolvimento, arquitetura modular, organização por camadas e foco em segurança, validação e documentação.

---

2. Descrição Completa do Projeto

A API permite o cadastro, atualização, listagem e remoção de entidades essenciais para o ecossistema de oportunidades sociais.
Também inclui autenticação, middleware de segurança, logs detalhados, documentação técnica e validações completas.

A arquitetura foi planejada para oferecer um backend robusto, reutilizável, escalável e fácil de integrar.

3. Funcionalidades Desenvolvidas

✔ CRUD completo de todas as entidades

- Categorias
- Organizações
- Oportunidades
  Pessoas
- Interesses (ligação entre pessoas e oportunidades)

✔ Autenticação e Segurança

- Login com JWT
- Middleware de autenticação
- Middleware global de erros
- Middleware de logs com cores e emojis
- Validações básicas e avançadas

✔ Banco de Dados

- SQLite com migrations e seeds
- Relacionamentos 1:N e N:N
- Diagrama ER inicial e final

✔ Documentação

- Documentação completa em Markdown
- Exemplos de requests/responses
- Regras de negócio
- Instruções de instalação e uso
- Endpoints detalhados

✔ Testes Manuais

- Testes de todos os endpoints no Insomnia
- Testes com cenários de erro e sucesso
- Testes de autenticação e tokens JWT
- Testes de relacionamento (Interesses + Oportunidades)

4. Tecnologias Utilizadas

Backend

- Node.js
  - Ambiente de execução JavaScript no lado do servidor.
  - Permite construir aplicações rápidas e escaláveis utilizando arquitetura assíncrona.

- Express.js
  - Framework minimalista para criação de APIs REST.
  - Facilita a definição de rotas, middlewares, tratamento de erros e organização do backend.

Autenticação & Segurança

- JWT (jsonwebtoken)
  - Biblioteca usada para gerar tokens de autenticação.
  - Permite que usuários façam login e acessem rotas protegidas de forma segura.

- bcryptjs (hash de senha)
  - Utilizado para **criptografar senhas** antes de salvar no banco.
  - Garante que nenhuma senha fique exposta em texto puro.

Validações

- express-validator
  - Conjunto de middlewares que valida entradas do usuário.
  - Evita que dados inválidos ou mal formatados cheguem ao banco de dados.

Banco de Dados

- SQLite
  - Banco de dados leve, baseado em arquivo.
  - Perfeito para projetos de extensão e ambientes com baixa complexidade.

- sqlite3 (driver)
  - Biblioteca que permite que o Node.js se comunique com o banco SQLite, executando queries e retornando resultados.

- Migrations/Seeds manuais
  - Migrations: criam e atualizam a estrutura do banco (tabelas, colunas, constraints).
  - Seeds: populam o banco com dados iniciais para testes.

Auxiliares

- Nodemon
  - Ferramenta que reinicia automaticamente o servidor sempre que um arquivo do projeto é alterado, acelerando o desenvolvimento.

- Middleware customizados (logger, errorHandler)
  - **Logger:** registra cada requisição, incluindo método, rota, status e tempo de execução.
  - **errorHandler:** captura erros e envia respostas padronizadas ao client

5. Arquitetura, Telas, Prints e Resultados

Arquitetura da Aplicação

```
/src
  /controllers     → Lógica de cada entidade
  /routes          → Arquivo de rotas REST
  /middlewares     → Auth, Erro, Logger
  /validators      → Regras de validação
  /database        → Migrations, seeds e conexão
/docs              → Documentação completa
README.md          → Guia principal do projeto
```

Fluxo da API

1. Cliente faz requisição →
2. Validação (express-validator) →
3. Autenticação (JWT) quando necessário →
4. Controller processa →
5. Banco de dados responde →
6. Middleware final formata resposta e logs

Demonstração (Prints)

- ## CRUD de Categorias

GET http://localhost:3000/api/categorias Send

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 253 Bytes Time: 14 ms

Response Headers 7 Cookies Results Docs { }

Copy


```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nome": "Curso",
      "descricao": "Cursos gratuitos e de curta duração"
    },
    {
      "id": 2,
      "nome": "Emprego",
      "descricao": "Vagas de emprego e estágios"
    },
    {
      "id": 3,
      "nome": "Mutirão",
      "descricao": "Ações sociais e serviços comunitários"
    }
  ]
}
```


- ## Criação de Oportunidade

POST http://localhost:3000/api/categorias Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format


```
{
  "nome": "Tecnologia",
  "descricao": "Cursos e eventos de tecnologia"
}
```


Status: 201 Created Size: 138 Bytes Time: 254 ms

Response Headers 7 Cookies Results Docs { }


```
{
  "success": true,
  "message": "Categoria criada com sucesso",
  "data": {
    "id": 4,
    "nome": "Tecnologia",
    "descricao": "Cursos e eventos de tecnologia"
  }
}
```


- ## Login funcionando

POST http://localhost:3000/api/pessoas/login Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format


```
{
  "email": "joao.silva@email.com",
  "senha": "senha123"
}
```

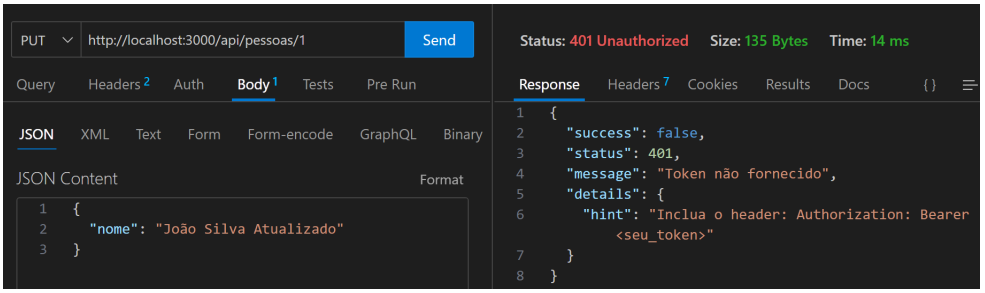

Status: 200 OK Size: 351 Bytes Time: 88 ms

Response Headers 7 Cookies Results Docs { }

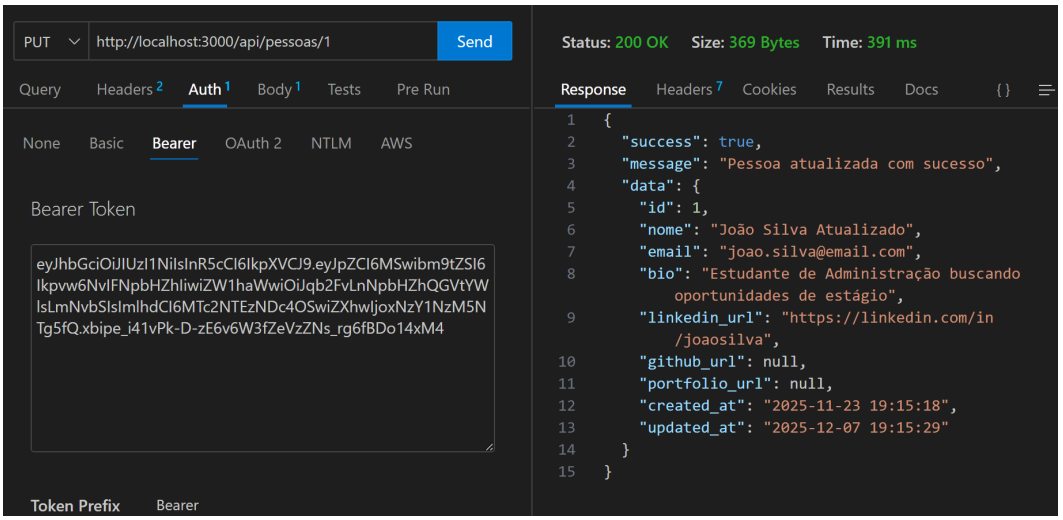
Copy


```
{
  "success": true,
  "message": "Login realizado com sucesso",
  "data": {
    "user": {
      "id": 1,
      "nome": "João Silva",
      "email": "joao.silva@email.com"
    },
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwibm9tZSI6Ikpvw6NvIFNpbHZhIiwiZW1haWwiOiJqb2FvLnNpbHZhQGVtYWlsLmNvbSIsImlhdCI6MTc2NTEzMzc2OCwiZXhwIjoxNzY1NzM4NTY4fQ.BLkGrifgE7lqCea9OR_CR8ALLUSNMPn1VmovNQ1Zn_0"
  }
}
```

- **Rota protegida exigindo token**



- **Logger funcionando no terminal**



6. Resultados Alcançados

- A API está **completamente funcional**.
- Todas as entidades possuem CRUD com validação.
- O sistema possui autenticação segura com JWT.
- A arquitetura respeita boas práticas de organização.
- Todos os endpoints foram testados e validados.
- Trello e GitHub estão atualizados com evidências completas.

7. Atividades realizadas por cada integrante

O projeto foi desenvolvido **individualmente**.
Todas as etapas foram realizadas por:

Leandro Mota Leal

ADS – Bloco Extensão Individual

---

8. Registro das Horas Trabalhadas

As horas foram registradas no formato exigido:
✔ Data do dia
✔ Tarefa executada
✔ Quantidade de horas
✔ Evidência anexada no Trello

Total do semestre entregue: 80 horas

Conferir TimeSheet Atualizado

---

9. Considerações Finais

O desenvolvimento da API permitiu consolidar conhecimentos essenciais em construção de serviços backend, arquitetura de software, APIs REST, autenticação, segurança e documentação técnica.

O principal desafio foi organizar corretamente a estrutura das entidades e implementar as regras de negócio, mas o resultado final demonstra um sistema sólido, escalável e pronto para ser integrado a aplicações reais.